

一些和画图无关的概念（对应 1-3 章，【不考】）

这门课只涉及功能性需求，不考虑实际的实现，“完美技术假设 Perfect Technology Assumption”。

获取信息的三个问题：跟谁收集？收集什么？怎么收集？

定义用例的三个技巧：用户目标 User Goal，增删改查 CRUD，事件分解 Event Decomposition。

Elementary Business Process: a task, performed by **one person**, in **one place**, at **one time**; “Meaningfully small”

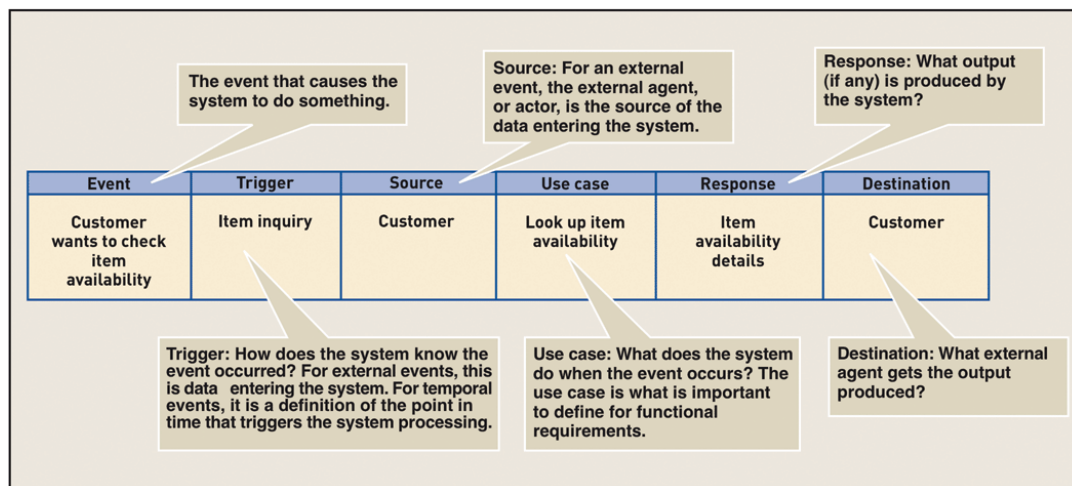
基本业务流程：一项由一个人在同一时间、同一地点执行的任务，“有意义的小”。

Use Case: an **EBP**, functions identified, the system performs, usually response to a request by a user; a verb + a noun.

用例：系统执行的**基本业务流程**，已确定的功能，通常是作为用户需求的响应。命名为“动词+名词”

Event Table: a catalog of use cases listed by event. Contains detailed information. After Event Decomposition.

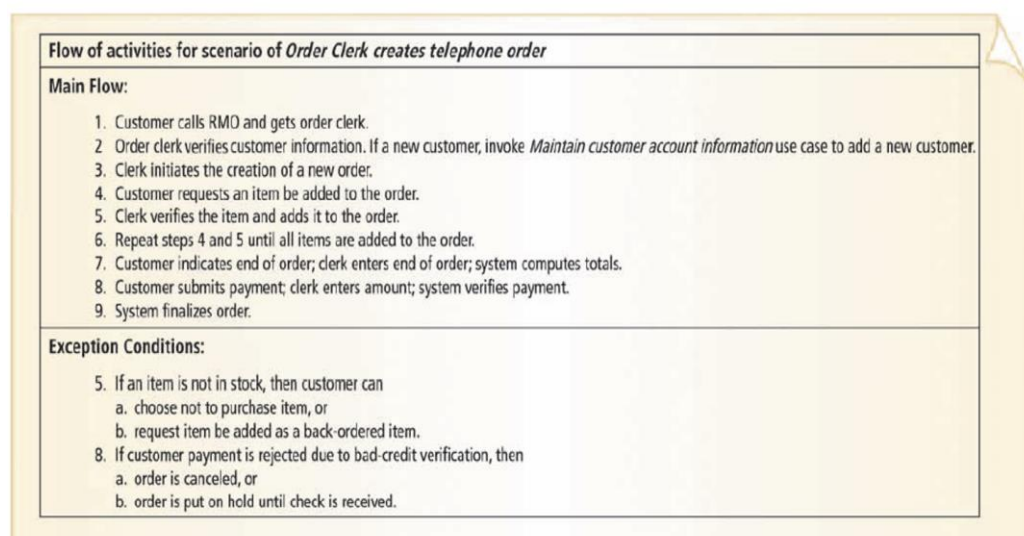
事件表：按照事件列出的用例目录，包含详细信息。事件分解后得到。



Use Case Descriptions: a description, detailed workflow, for each of the use cases. including Brief description, Intermediate description, Fully developed description.

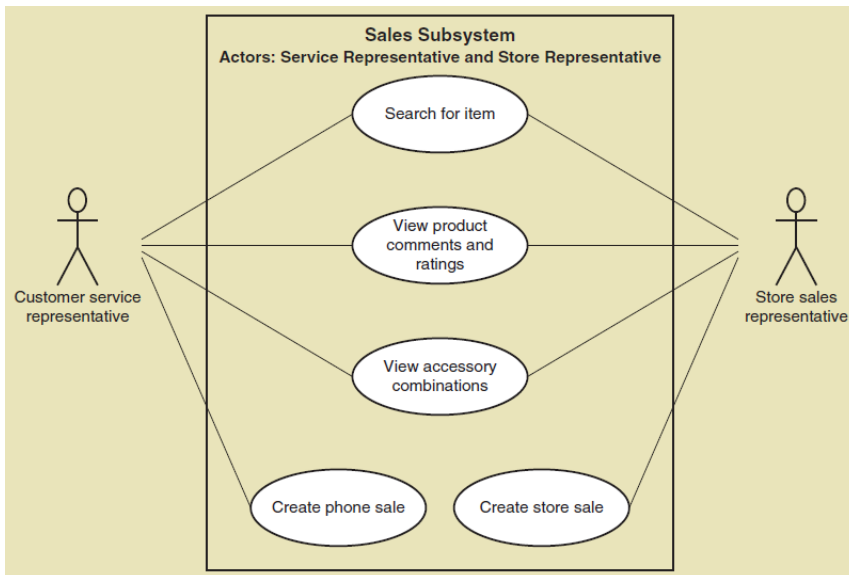
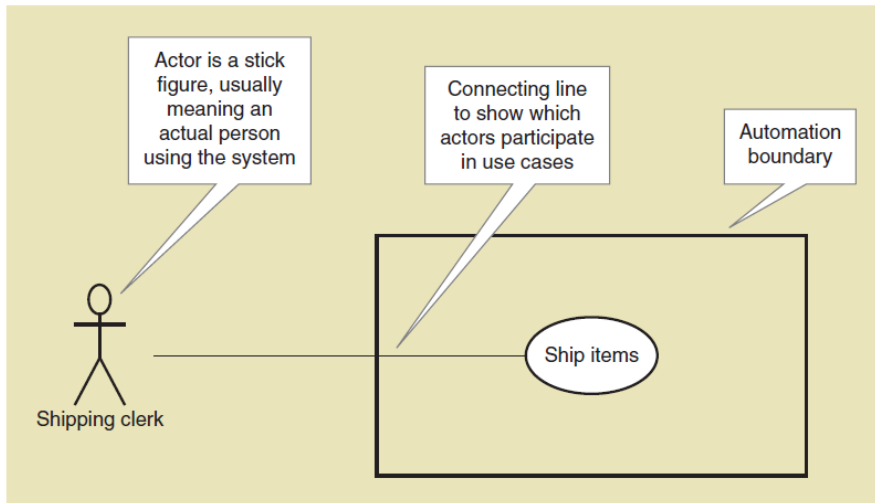
用例描述：用例详细流程的描述，有简要描述，中间描述（一般都是这个），全面描述三类。

Intermediate Description

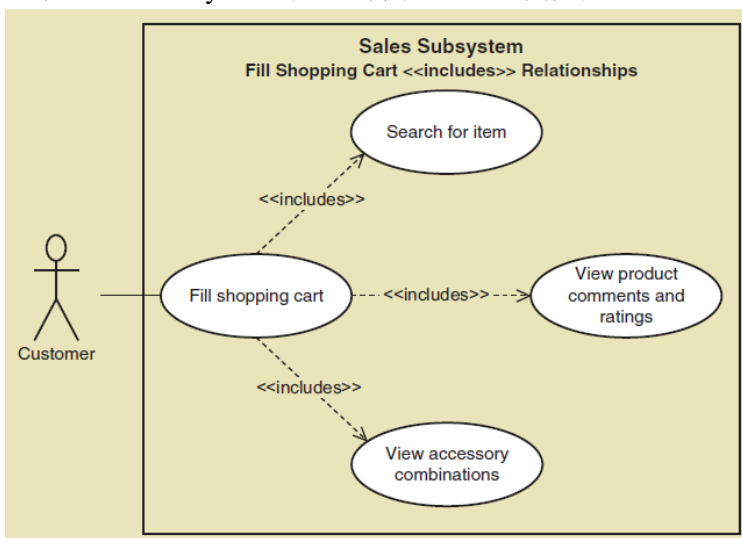


用例图 Use Case Diagram 【大概率题目会给出，但不用画】

Actor: End User 最终用户。 Automation Boundary: 用户和系统的边界。 线: 连接用户和用例。



细节: XX Subsystem 下面注明了 Actors 都有哪些。



细节: 包含关系, 箭头指向子程序, 用虚线, 标注<<include>>

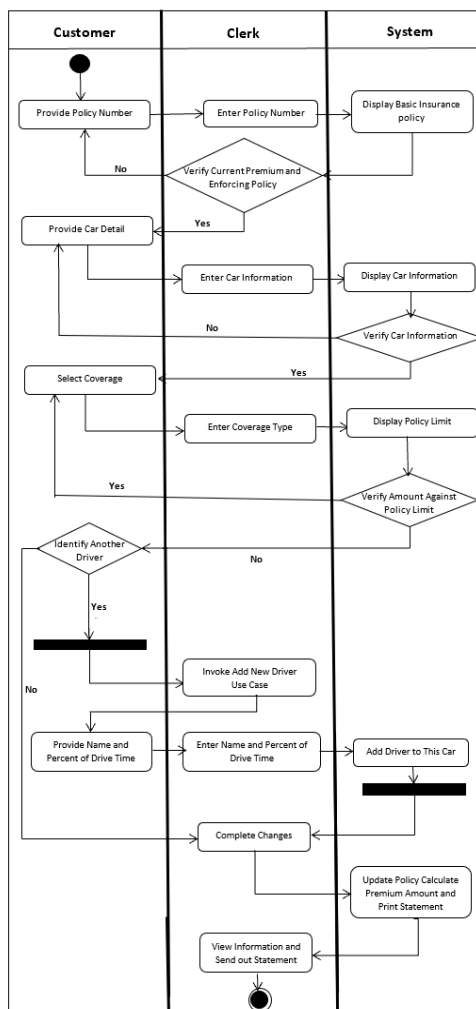
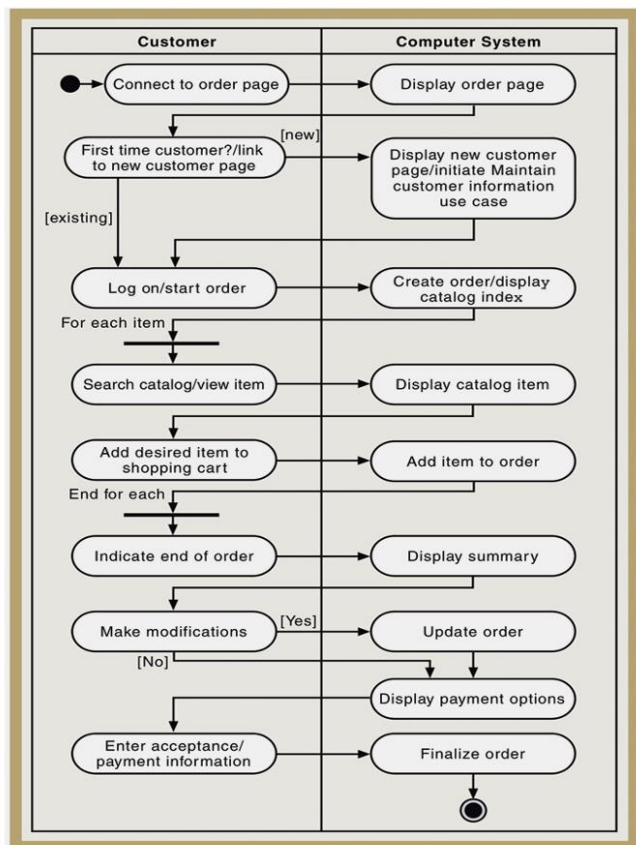
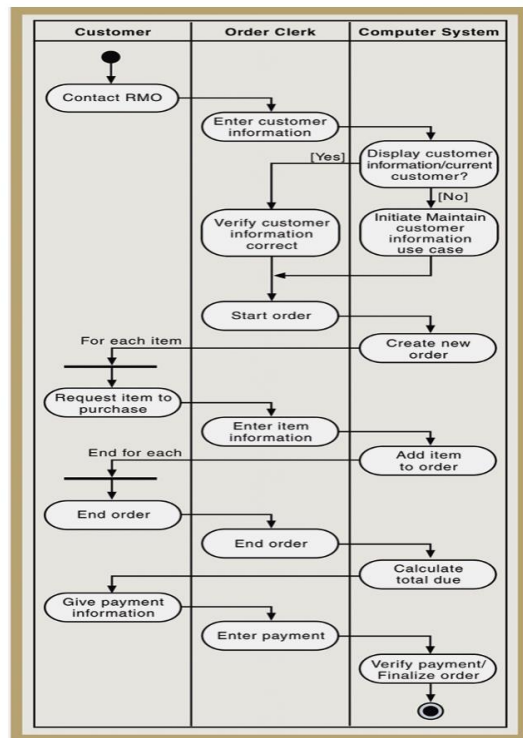
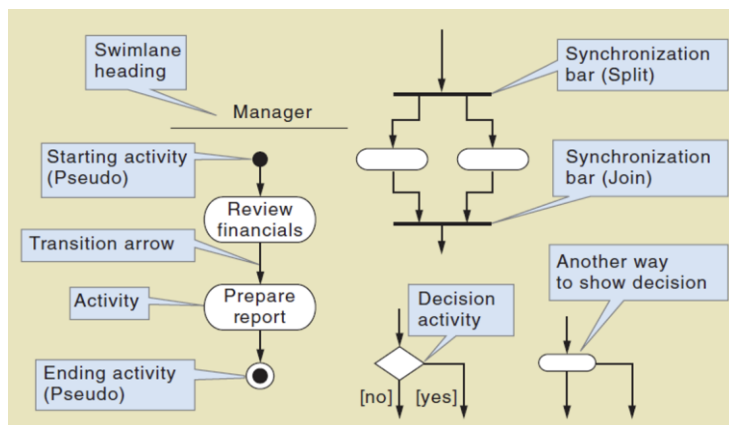
活动图 Activity Diagram【大概率考】

记录每个用例或场景的业务流程工作流。**用例描述的补充**，不是用例！

这里可能会涉及多个人，例如下右图，描述的是一个电话购物场景，customer 和 clerk。由于只有 clerk 和系统有交互，所以在后面的 SSD（系统时序图）只有 clerk 一个 actor，没有 customer。

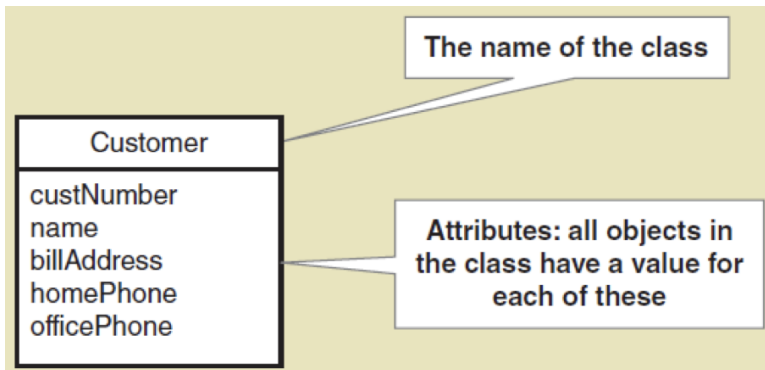
画图细节：

1. 开始标志只有实心点，结束标志实心点外要画个圈；
2. 迭代/同步开始和结束需要画横着的粗黑线；
3. 用圆角矩形来表示活动，注意不是用例（所有图中，只有用例图中的圆圈内才表示用例）。



【分析】类图 [Analysis] Class Diagram 【一定会考】

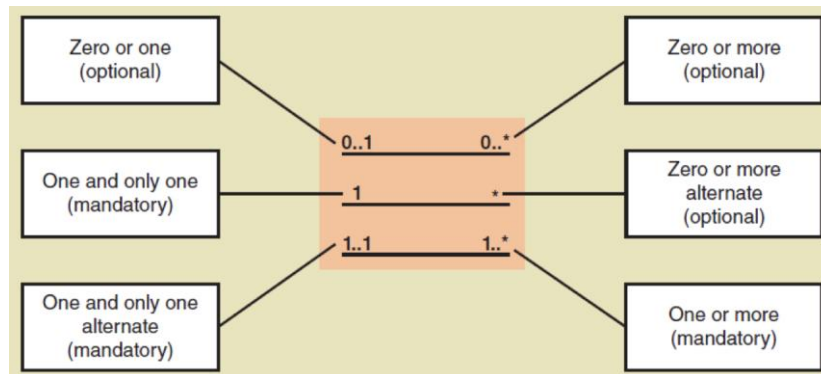
事物被建模为 Domain Class 或 Data Entity, 用名词技巧 Noun Technique 确定哪个名词是 Data Entity 还是 Attribute, 或者是无关名词。排除同义词、已定义信息产生的输出、已定义信息的输入。



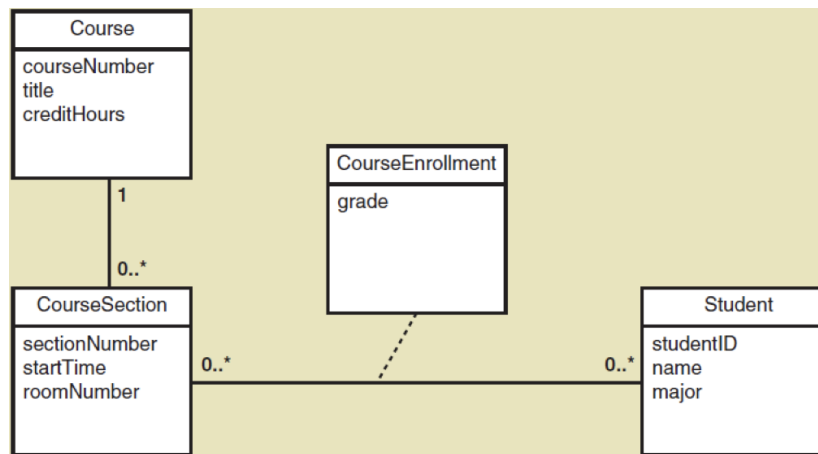
细节：类名大写，属性小写驼峰。

Data Entity 类需要标识符或键，Domain Class 可以不需要。

类之间有关联 Association 关系。如果最少有一个，关联是强制性的。

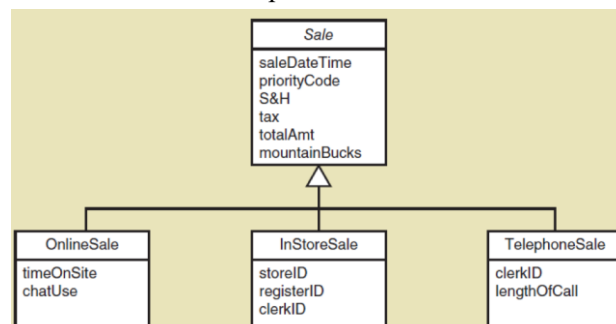


多对多关联需要一个关联类，用于建模这两个实体，并存储有关信息。



目的：符合数据库设计的第二范式，数据库表中的所有非主键字段必须完全依赖于表的主键。

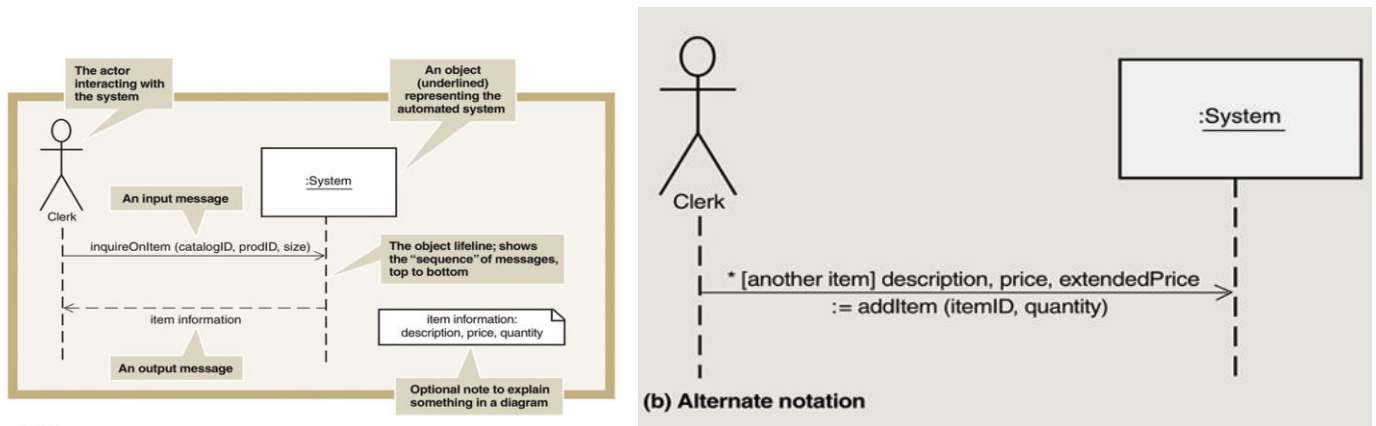
继承关系中，子类 Subclass 用三角箭头指向超类 Superclass。子类会继承超类的关联关系。抽象类用斜体表示。



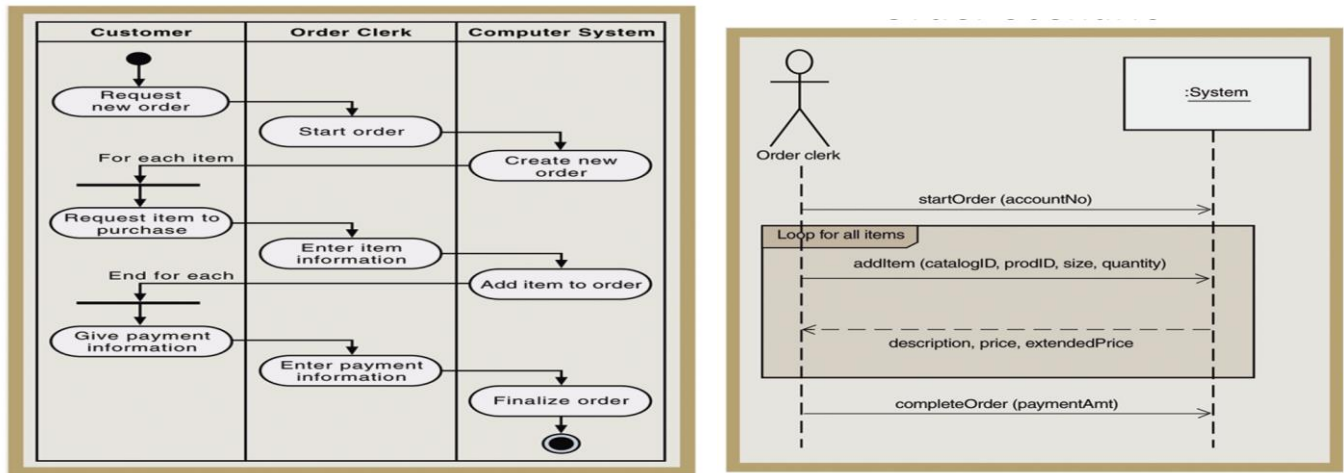
对于整体和部分的的关系，聚合 Aggregation 表示组成的部分可以单独存在，不可以被移除和替换，用空心菱形符号表示。构成 Composition 表示组成的部分不可以再被移除，用实心菱形箭头表示。这个应该不考。

【分析】系统时序图 [Analysis] System Sequence Diagram 【一定会考】

将系统视为黑箱，模拟用例或者某个场景的输入输出消息需求，以信息形式显示互动顺序。



对于如下的活动图，由于是 clerk 在和系统交互，所以 SSD 只能写 clerk:

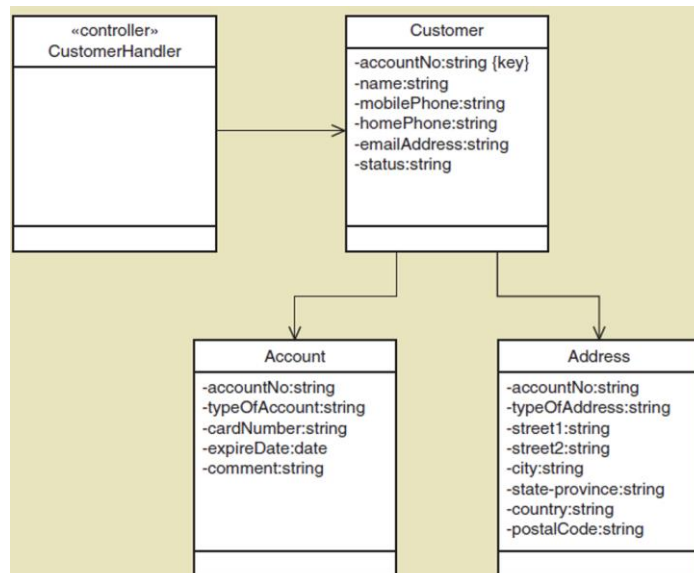


画图细节：系统是一个类，作为系统黑箱，名字左边加冒号，底部下划线。方法和返回值小写驼峰。Frame 左上角是缺右下角的矩形。

【设计】初步设计类图 First-Cut Design Class Diagram 【一定会考】

Controller 处理从视图层到问题域层类的所有消息，接收所有消息并充当交换机，将消息引导到问题域。

对于某个用例，在设计类图阶段，首先需要选择所需的类。之后，详细说明属性的可见性（public/private）和数据类型。之后，添加导航可见性，添加箭头（a->b，a 调用了 b）。最后需要添加控制器类。



【设计】设计系统时序图 Design System Sequence Diagrams 【一定会考】

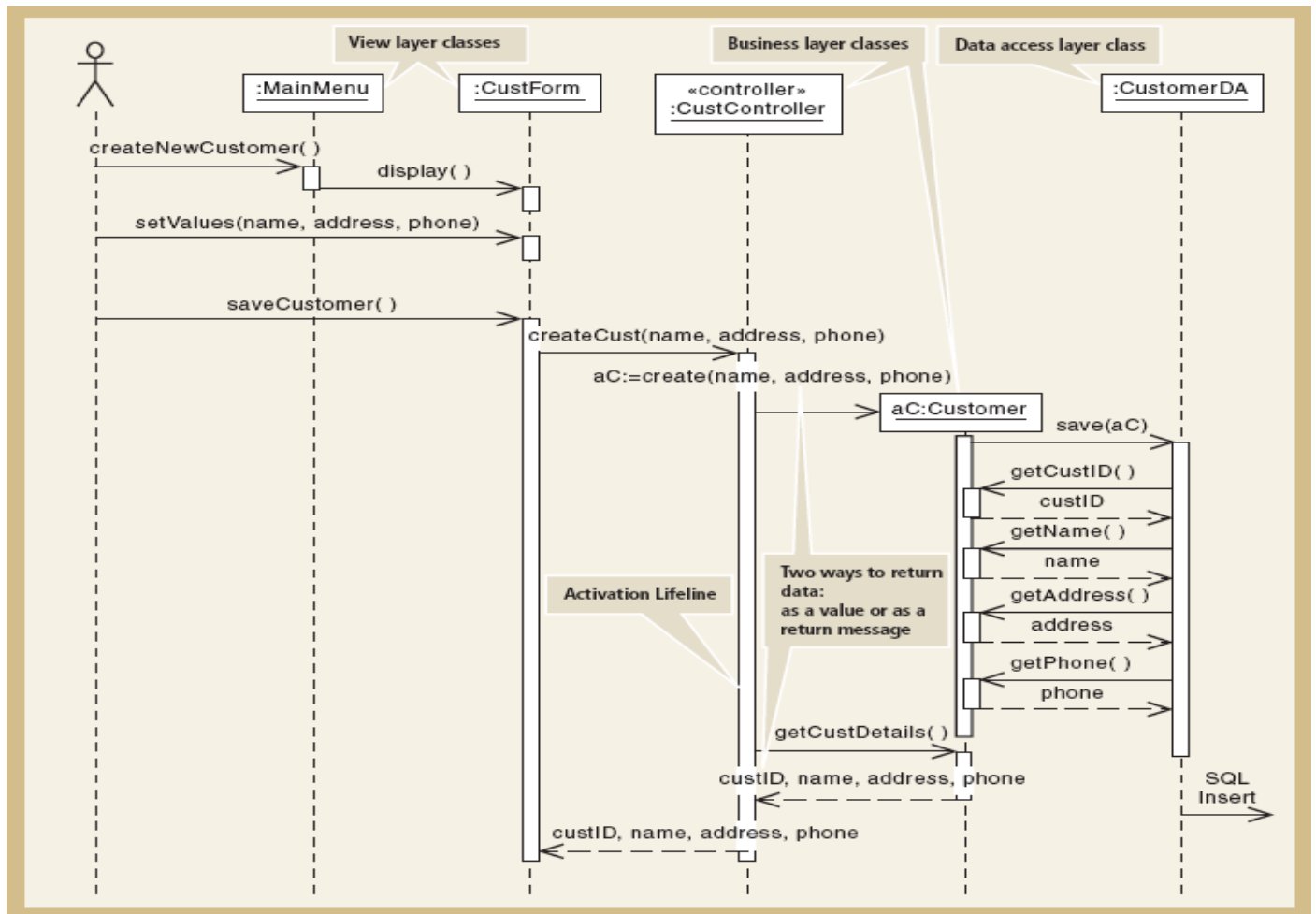
初步设计类图：选择所需类，详细说明属性，添加导航可见性，添加控制器类。

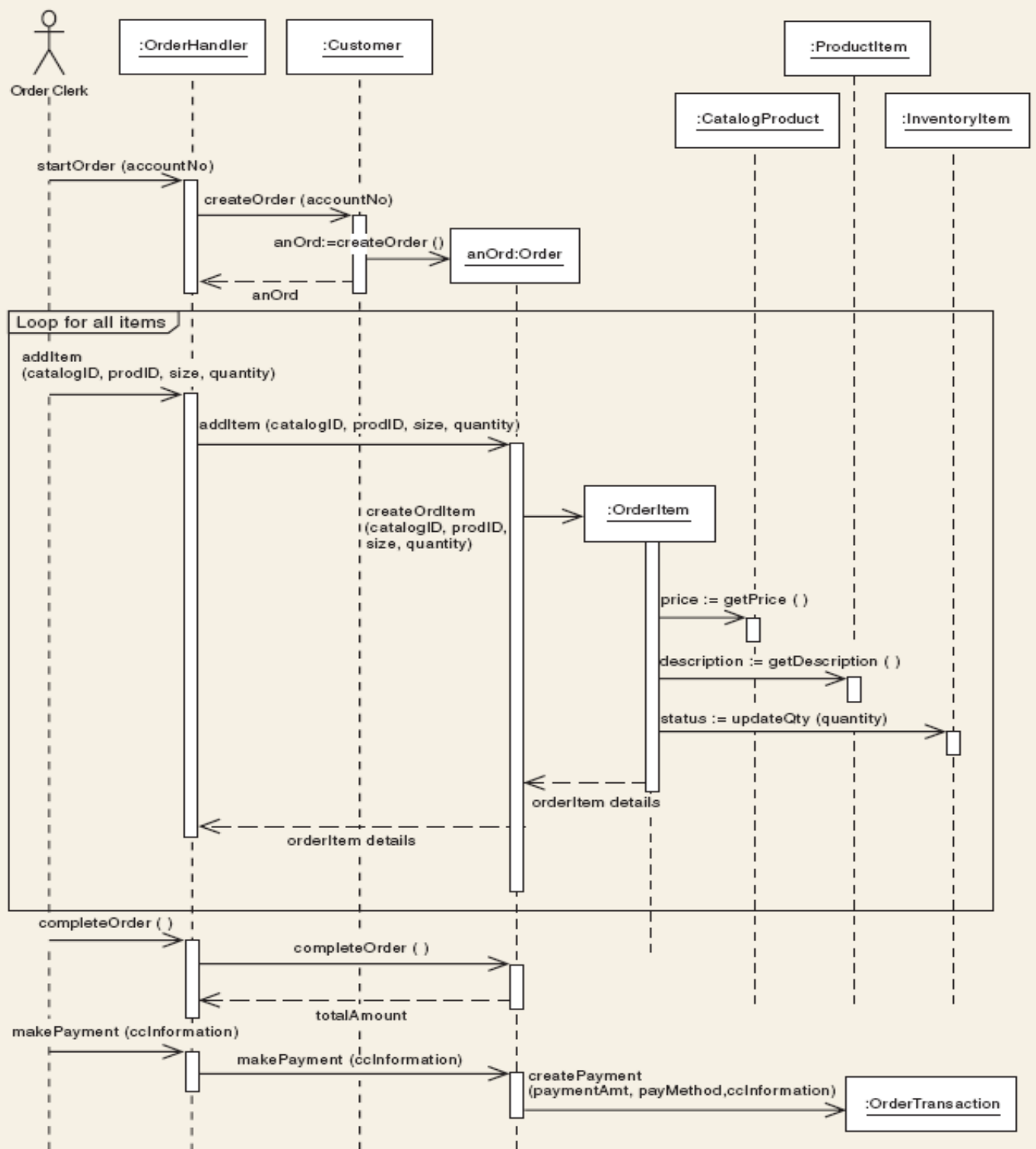
视图层：与用户交互，添加表单或页面。视图类不需要知道领域层或数据访问层。

数据访问层：存储和检索数据库数据。 对象：类的实例化，用 aClassname 或者 anClassname 标注，仍然用矩形方框标注，规则：aClassname:Classname，实例名称 aClassname 中的类名可以简写。

首先用户要访问一个页面，例如主页或者表单，属于 View Layer Classes。之后页面发送请求给 Controller，注意 Controller 需要标注为<<controller>>。控制器会按照需求创建对象。它们都属于业务逻辑层的类。

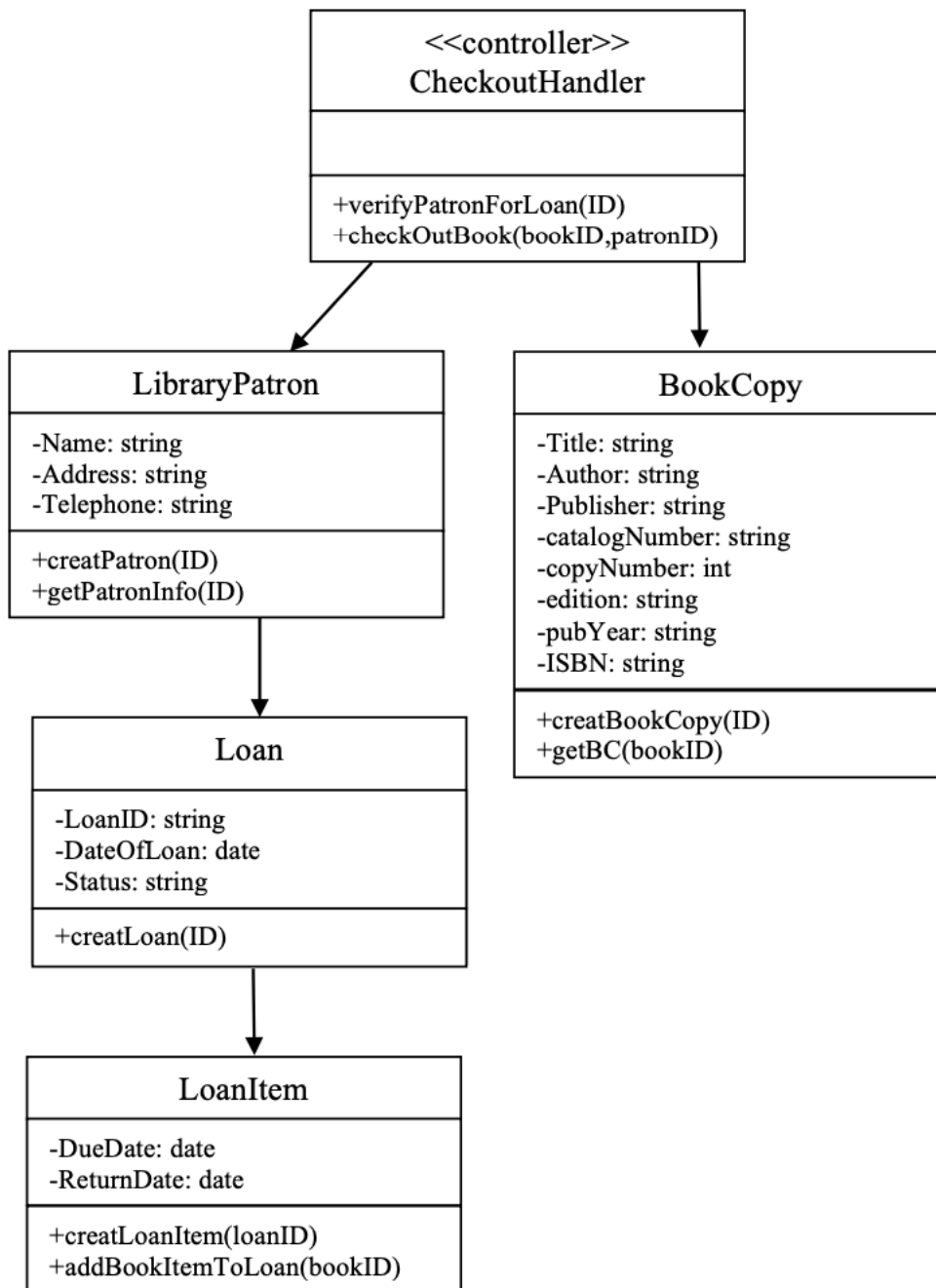
创建对象、查询等增删改查（CRUD）肯定会涉及到数据操作，所以需要数据访问层，也就是 DA。这里 DA 不用标注实际的 SQL 操作。





【设计】完整设计类图 Complete Design Class Diagram [应该不会考]

根据 Design SSD 添加方法。因为只是在 First-cut 的基础上用添加方法，因此大概率不会考。
 这里的方法都是以传 ID 作为参数。



【题型猜测】

按照顺序，所有可能会考的图如下：

•【分析】阶段：

Use Case Diagram

Activity Diagram

Analysis Class Diagram

Analysis System Sequential Diagram

•【设计】阶段：

First-cut Design Class Diagram

Design System Sequential Diagram

Complete Design Class Diagram

【一般会给出，小概率会需要画】

【应该只考几个用例】

【应该考整体的类图】

【重点应该是后续的 design SSD】

【应该只考某个用例】

【应该只考某个用例】

【应该不会单独考】